Supplement for

Process Bigraphs and the Architecture of Compositional Systems Biology

Eran Agmon *

Ryan K. Spangler

Center for Cell Analysis and Modeling,
University of Connecticut Health,
Farmington, Connecticut, USA

March 3, 2026

Contents

1 Introduction **2**

2 Vivarium 2.0 software suite **2**

3 Process–Bigraphs Semantics **3**

3.1 Mathematical preliminaries . . . 3
3.2 Serialized and human-readable forms . . . 3
3.3 Types, schema trees, and state trees . . . 4
3.3.1 Type engine and type registry . . . 5
3.4 Links, ports and wiring . . . 5
3.5 Processes and delta semantics . . . 6
3.5.1 Process nodes and the link registry . . . 6
3.5.2 Deltas and type-directed application . . . 7
3.5.3 Single-process update protocol . . . 7
3.6 Explicit definition of a Process–Bigraph . . . 8
3.7 Composites . . . 8
3.8 Orchestration . . . 9
3.8.1 Event times and schedule . . . 10
3.8.2 Small-step orchestration semantics . . . 10
3.8.3 DAG-structured discrete workflows . . . 11
3.8.4 Recursive composition . . . 11
3.9 Emitters and result capture . . . 11

*Correspondence: agmon@uchc.edu

4 Spatio–Flux Instantiation **12**
4.1 Spatio–Flux processes . . . 12
4.1.1 Local metabolic processes . . . 12
4.1.2 Field movement . . . 14
4.1.3 Particle movement processes . . . 16
4.1.4 Particle–field adapters . . . 18
4.1.5 Structural and Boundary – Graph rewrites . . . 19
4.2 Particle–COMETS composition . . . 22

1 Introduction

This supplementary materials document for "Process Bigraphs and the Architecture of Compositional Systems Biology" provides the technical underpinnings of the Process–Bigraph framework introduced in the main text. Our goals are to (i) make the semantics of Process–Bigraphs precise, (ii) connect these abstract definitions to concrete Python classes in the Vivarium 2.0 software stack, and (iii) illustrate the framework on a fully worked example (Spatio–Flux).

We proceed with the following sections:

- Section 2 introduces the Vivarium 2.0 software suite, the core libraries used in this work, and the main runtime classes we refer to in the rest of the supplement.
- Section 3 develops the formal, schema-based semantics of Process–Bigraphs in an implementation-agnostic way, but with direct correspondence to the classes and type registries provided by the software libraries.
- Section 4 gives a detailed Spatio–Flux instantiation of these semantics, showing how field dynamics, stochastic/deterministic particle movement dynamics, and coupled particle–field metabolism are represented as Process–Bigraphs and executed by the Vivarium 2.0 stack.

2 Vivarium 2.0 software suite

The Vivarium 2.0 software suite provides the reference implementation of Process–Bigraphs used in this paper. It consists of three core libraries and an application repository:

- `bigraph-schema`: a foundational library that defines a compositional type system and hierarchical data structures using JSON-based schemas. It provides the *type engine* (e.g. `Core`) and a global `type_registry` that together implement: (i) schema compilation to path-addressed schema trees, (ii) validation of state trees, and (iii) type-specific update operators `apply` used in the delta semantics of Section 3.
- `process-bigraph`: the dynamic core of the framework. It defines:
  - the `Process` and `Step` classes, which implement typed *process nodes* with input/output ports, configuration, and an `address` used to look up their implementation;
  - the `Composite` class, which encapsulates an entire process–bigraph as a single process with an explicit interface;
  - the event-scheduling and orchestration logic that maintains a global time, an update schedule, and the repeated evaluation of process handlers.
- `bigraph-viz`: a visualization library that parses the JSON representation of a process–bigraph and renders it as a graph.

In addition to the core libraries, we use a domain-specific application repository:

- `spatio-flux`: a complete multiscale example built on `process-bigraph` and `bigraph-schema` that defines a `Process` and `Composite` subclasses for: metabolic processes, field dynamics, particle dynamics, and coupled particle–field interactions. The repository includes an extended test suite that runs dozens of process–bigraph documents, some of which are referenced in Section 4 as concrete instances of the abstract semantics.

All source code, documentation, tutorials, and examples are available on the Vivarium Collective GitHub:

- `process-bigraph`: `https://github.com/vivarium-collective/process-bigraph`
- `bigraph-schema`: `https://github.com/vivarium-collective/bigraph-schema`
- `bigraph-viz`: `https://github.com/vivarium-collective/bigraph-viz`
- `spatio-flux`: `https://github.com/vivarium-collective/spatio-flux`

3 Process–Bigraphs Semantics

This section develops the formal, schema-based semantics of Process–Bigraphs, expressed in a set-theoretic, function-based language. We proceed from mathematical preliminaries to types, states, wiring, processes, composites, and orchestration. Section 4 then mirrors this general account with a Spatio–Flux-specific instantiation, so that readers can move back and forth between abstract definitions and concrete examples.

3.1 Mathematical preliminaries

A bigraph is organized as a hierarchical *tree* (or, more generally, a forest) whose nodes are addressed by finite paths. This representation matches naturally with JSON: objects correspond to nodes with children keyed by field names, arrays correspond to nodes with children keyed by integer indices, and following a sequence of keys or indices selects a unique location in the structure.

Let $\mathcal{M}$ be the set of *marks* (atomic path components), and let

$$\mathcal{P} = \mathcal{M}^*$$

be the set of finite paths, implemented as lists of marks (e.g. `["organism", "cell", "mass"]`).

Any JSON-like hierarchical structure induces a tree of such paths: objects and arrays correspond to internal nodes, and primitive values correspond to leaves. In this supplement, both schemas and states are treated as partial maps from paths in $\mathcal{P}$ to types or values, respectively.

We can fix:

- a set of types $\mathcal{T}$,
- a set of values $\mathcal{V}$,
- a subset of paths $\mathsf{StorePath} \subseteq \mathcal{P}$ that designate locations where concrete values may be stored.

3.2 Serialized and human-readable forms

A process–bigraph may appear in three equivalent textual forms (Fig. 1):

1. a JSON-like serialized document, which is the canonical stored format;
2. a reduced tree notation, used in this supplement for readability;
3. a linearized one-line syntax using bracketed annotations.

These formats differ only in presentation. JSON and the linearized syntax can be parsed by the `Core` engine into the same in-memory executable composite, and round-trip conversion between

them is deterministic and lossless. The reduced tree notation is a human-readable rendering of the same structure, used here for clarity.

```
{
  "schema": {
    "stores": {
      "store": "float"
    },
    "processes": {
      "proc": {
        "inputs": ["store"],
        "outputs": ["store"]
      }
    }
  },
  "state": {
    "stores": {
      "store": 0.0
    }
  }
}
```

```
schema:
  stores:
    store: float
  processes:
    proc:
      inputs: [store]
      outputs: [store]
state:
  stores:
    store: 0.0
```

Figure 1: **Equivalent representations of a simple process–bigraph.** *Left:* JSON-like serialized document. *Right:* Reduced tree notation used throughout this supplement. The same structure can also be written in linearized form as: `schema[stores[store:float]|processes[proc:inputs[store]|outputs[store]]]|state[stores[store:0.0]]`.

3.3 Types, schema trees, and state trees

A *schema tree* is a partial map

$$\Sigma : \mathcal{P} \rightharpoonup \mathcal{T},$$

assigning a type to each path in a JSON-like hierarchical structure. Each $\Sigma(p)$ specifies the type governing the subtree rooted at p. Composite types appear at internal nodes and constrain the structure of their children. Leaf types specify primitive values such as numbers, strings, or booleans. A schema tree therefore determines field types, array shapes, port signatures, and the types of all `process` classes.

A *state tree* is a partial map

$$x : \mathcal{P} \rightharpoonup \mathcal{V},$$

assigning concrete values to paths. Leaves at designated store paths correspond to the *stores* of the system. Internal nodes may be objects or arrays that organize the hierarchy.

A state tree x is *well-formed* for a schema tree Σ when, for every path p where both are defined, the value $x(p)$ is valid for the type declared at $\Sigma(p)$. We write this as the typing judgment

$$\Sigma \vdash x : \mathsf{State},$$

which is read as: under the schema Σ, the state tree x is a well-formed state.

Figure 2 illustrates a simple schema tree (left) and a matching state tree (right), showing how the schema constrains the structure and types of the corresponding concrete values.

```
organism:
  cell:
    volume:  float (um3)
    mass:  float (pg)
  environment:
    temperature:  float (K)
    ph:  float
```

```
organism:
  cell:
    volume:  1.2
    mass:  0.85
  environment:
    temperature:  310.0
    ph:  7.2
```

Figure 2: **Example of schema and state trees in reduced textual format.** *Left:* schema hierarchy with unit-annotated leaf types. *Right:* matching state supplying concrete values consistent with the schema.

3.3.1 Type engine and type registry

The type engine (e.g. `bigraph-schema Core`) maintains a global `type_registry`, written R_T. This registry defines all available types and their semantics, including:

- validation and parsing of values,
- an update operator apply_τ for each type $\tau \in \mathcal{T}$,
- serialization and deserialization methods.

Given a schema tree Σ and state tree x, the type engine checks that the typing judgment $\Sigma \vdash x : \mathsf{State}$ holds.

3.4 Links, ports and wiring

In `bigraph-schema`, a *link* is a node whose schema specifies a set of typed input and output ports. Figure 3 shows a typical example: the schema for `"mm_process"` declares two input ports (`"substrate"`, `"enzyme"`) and one output port (`"product"`), each annotated with a type.

Formally, for a link at path $p \in \mathcal{P}$, the schema tree Σ determines:

- its interface, with input ports $P_{\text{in}}(p)$ and output ports $P_{\text{out}}(p)$,
- for each port ℓ, the declared type $\tau_\ell = \Sigma(p.\ell)$.

The corresponding state-tree representation is shown in Fig. 3 (right). A link node in the state tree x stores:

- `inputs`: mapping input-port names to the state paths where their values are read,
- `outputs`: mapping output-port names to the state paths where their updates are written,
- `config`: process-specific parameters.

For example, the `mm_process "substrate"` input port is wired to `["state","substrate_conc"]`, and `"product"` is wired to `["state","product_conc"]`.

From these `inputs` and `outputs` fields the global wiring pattern is obtained. The wiring map is therefore not stored separately: it is *derived* from the collection of link nodes in x.

Formally, collecting these entries across all link paths yields:

$$W : (\mathsf{LinkPath} \times \mathsf{PortName}) \to \mathcal{P},$$

where $W(p, \ell)$ returns the state-tree path connected to port ℓ of the link at p, and $\mathsf{LinkPath}$ is the set of paths whose schema entries denote links or their specializations (e.g. `process`, `step`, `composite`).

Wiring is *well-typed* when each port is connected to a state path whose schema type matches the port's declared type. For instance, the `"substrate"` input of the Michaelis–Menten process must be wired to a store typed as `float (mM)` in Σ.

```
mm_process:
  _type: process
  _inputs:
    substrate: float (mM)
    enzyme: float (mM)
  _outputs:
    product: float (mM)
  config:
    kcat: float (1/s)
    Km: float (mM)
  address: protocol
  interval: float (s)
```

```
mm_process:
  inputs:
    substrate: [state, substrate_conc]
    enzyme: [state, enzyme_conc]
  outputs:
    product: [state, product_conc]
  config:
    kcat: 1.2
    Km: 0.5
  address: local:MichaelisMenten
  interval: 0.1
```

Figure 3: **Example of a process schema and matching state in reduced textual format.** *Left:* schema for a Michaelis–Menten process, including typed ports, configuration fields, and metadata. *Right:* corresponding state that binds ports to store paths, assigns numeric parameters, and selects a concrete implementation via the `address` field.

3.5 Processes and delta semantics

3.5.1 Process nodes and the link registry

`process-bigraph` extends the generic typed-link structure by introducing `Process`, `Step`, and `Composite` as *specialized* link types. Like all links, a process node inherits typed input and output ports and participates in the global wiring map W; it additionally carries the fields needed for runtime execution.

These fields are:

- an `address` identifying a handler class in the `link_registry`,
- a typed `config` subtree describing its parameters,
- an `interval` declaring its nominal time advance,
- typed input/output port declarations inherited from the schema.

The *link registry* $R_L : \mathsf{Address} \to \mathsf{HandlerClass}$ maps each address to a concrete handler class. Initialization instantiates the handler using the node's `config` and stores the instance at the same path p_{proc} in the in-memory state tree. Thus the schema determines what kind of process may appear, while the handler instance determines how that process behaves at runtime.

Process interfaces as typed function signatures. At the schema level, every process node specifies a *typed interface* describing what it consumes and what it produces. This can be written as a function signature

$$\mathsf{process}_{p_{\text{proc}}}\big(\text{config}^{\tau_c}\big) : \big(\text{in}_1^{\tau_{i1}}, \text{in}_2^{\tau_{i2}}, \dots\big) \;\longrightarrow\; \big(\text{out}_1^{\tau_{o1}}, \text{out}_2^{\tau_{o2}}, \dots\big).$$

This signature expresses the process contract: its configuration space τ_c, the types of values it may read, and the types of values it may update.

Processes as delta generators. At runtime, the instantiated handler implements this signature via an `update` method. Abstractly, this has the form

$$\mathsf{update} : \big(\text{config}^{\tau_c}, \text{in}_1^{\tau_{i1}}, \dots\big) \;\longrightarrow\; \Delta,$$

returning a *composite delta* Δ, a tree-structured object whose shape mirrors the process's output schema and whose leaves are typed deltas associated with individual output ports. In this way, the schema-level view (typed function signature) and the runtime view (delta-producing handler) align: both describe the same transformation, one abstractly and one operationally.

3.5.2 Deltas and type-directed application

For each type $\tau \in \mathcal{T}$, the type engine defines a type-specific update operator

$$\mathsf{apply}_\tau : \mathcal{V}_\tau \times \Delta_\tau \to \mathcal{V}_\tau,$$

where Δ_τ is the class of deltas valid for τ. Deltas describe *how* a value should change. Each type determines the shape of its permissible deltas, how to apply them, and what counts as a well-formed update.

We distinguish three broad categories of deltas.

1. Primitive deltas. Primitive deltas are the *leaf-level* updates applied to values of non-structured types. For such leaf types, a primitive delta typically encodes a direct change — such as an increment, logical update, or replacement. Applying a primitive delta updates the associated store value directly:

$$x' = \mathsf{apply}_\tau(x, \delta).$$

2. Composite deltas. Structured types (records, arrays, maps) carry *tree-structured* deltas that mirror the shape of the composite value. A composite delta may contain primitive deltas at leaves, nested composite deltas at interior nodes, and optionally structural edit instructions. A process's `update` method usually returns such a delta tree, whose top-level branches correspond to its output ports. After wiring distributes these deltas to the appropriate state paths, each leaf is applied using the appropriate apply_τ operator.

3. Structural deltas. Structural deltas modify the *shape* of either the state tree or the schema tree. They include:

- **insert**: add a new subtree,
- **delete**: remove a subtree,
- **move**: relocate a subtree,
- **rewrite**: modify fields, types, shapes, or metadata,
- **rewire**: modify `inputs`/`outputs` so ports point to different stores.

Since processes, ports, schedule entries, and wiring tables all appear as subtrees, these edits naturally express graph rewrites such as division, merging, engulfment, or spawning. They require no special orchestration rules: they are applied through the same type-directed delta semantics as numerical updates.

3.5.3 Single-process update protocol

A single update step for a process located at path p_{proc} unfolds as follows.

1. Input projection. For each input port ℓ, the wiring map W specifies the source state path. The runtime gathers the current input values:

$$x_{\text{in}}(\ell) = x\big(W(p_{\text{proc}}, \ell)\big).$$

2. Evaluation. Given the timestep Δt chosen by the orchestration policy (Section 3.8), the handler instance computes

$$\delta = \mathsf{update}_{p_{\text{proc}}}(x_{\text{in}}, \Delta t),$$

yielding a composite delta whose fields mirror the process's output ports.

3. Application. For each output port ℓ, let $q = W(p_{\text{proc}}, \ell)$ be the target state path and let $\tau = \Sigma(q)$ be its declared type. The type engine then applies the delta using the type-directed rule

$$x'(q) = \mathsf{apply}_\tau\big(x(q),\, \delta(\ell)\big).$$

Applying all such deltas yields a new state tree x' that remains well-typed with respect to the schema Σ.

In summary, processes behave as typed function signatures at the schema level, handler instances with update methods at runtime, and generators of typed deltas that the type engine applies uniformly across numerical, structural, and graph-rewrite updates. This unification is what enables process bigraphs to support heterogeneous modeling methods and dynamic structural change within a single execution semantics.

3.6 Explicit definition of a Process–Bigraph

We can now summarize the core ingredients of the framework as a single structure.

A *Process–Bigraph* is a tuple

$$B = (\Sigma, x, R_T, R_L),$$

consisting of:

1. $\Sigma : \mathcal{P} \rightharpoonup \mathcal{T}$, a schema tree assigning a type to each path in a JSON-like hierarchy;
2. $x : \mathcal{P} \rightharpoonup \mathcal{V}$, a state tree assigning concrete values to paths, including the `inputs`, `outputs`, and handler instances stored at link and process nodes;
3. R_T, the type registry providing validation and type-directed update operators apply_τ for each type τ;
4. R_L, the link registry mapping process addresses to handler classes, from which configured process instances are created and stored in x.

A global wiring map W is *derived* from the `inputs` and `outputs` fields stored in the state tree x for each link node.

These components must satisfy the following consistency conditions:

- **Type correctness:** The typing judgment $\Sigma \vdash x : \mathsf{State}$ holds: whenever both $\Sigma(p)$ and $x(p)$ are defined, $x(p)$ is valid for the type declared at $\Sigma(p)$.
- **Well-typed wiring:** For every link node and port ℓ, the schema type declared for that port matches the schema type at its destination path obtained from the derived wiring map W.
- **Well-formed processes:** Each process node in `LinkPath` has a valid `address` pointing to a handler class in R_L, and the state tree contains a corresponding configured handler instance stored at the same path.

A Process–Bigraph is therefore a fully typed, wired, and executable structure, ready to be advanced in time by the orchestration semantics described in Section 3.8.

3.7 Composites

A `Composite` is a specialized `process` that serves as the main simulation engine for `process-bigraph`. Its configuration encodes an entire process–bigraph — a schema tree, a state tree, and the registries needed to interpret them — so that a composite behaves as a full, self-contained simulation when executed. An example is shown in Fig. 4.

A composite is also a process, that connect its internal process–bigraph to an external state. It does this with two addition attributes:

- declares an `interface`, consisting of typed input and output ports just like any other process node;
- declares a `bridge` mapping these external ports to internal state-tree paths within the embedded bigraph.

The interface specifies the composite's external API, while the `bridge` acts as the inward-facing wiring. The type of each interface port must match the schema type at its internal bridge destination.

At runtime, the composite's configuration stores:

- the embedded schema tree Σ_{state},
- the embedded state tree x_{state}, including all internal link and process nodes,
- the global `type_registry` R_T,
- the global `link_registry` R_L, providing handler classes for processes.

As in the top-level case, the internal wiring pattern is not stored separately: it is *derived* from the `inputs` and `outputs` fields of link nodes within x_{state}.

Operationally, a composite C is identified as

$$C = \big(\Sigma_{\text{state}},\ x_{\text{state}},\ R_T,\ R_L,\ \mathsf{bridge},\ \mathsf{interface}\big),$$

with internal wiring implicitly determined by x_{state}.

Under orchestration (Section 3.8), a composite behaves as a standard process externally: values arriving at its interface ports are routed inward through the `bridge`, internal processes advance according to their scheduling rules, and outgoing deltas are returned through the external ports. Composites thus provide hierarchical modularity: externally a single process, internally a complete process–bigraph.

```
process1:
  outputs:
    port1:  [store1.1]
    port2:  [store1.2]

process2:
  outputs:
    port1:  [store1.1]
    port2:  [store1.2]

bridge:
  inputs:
    port1:  [store1.1]
    port2:  [store1.2]
```

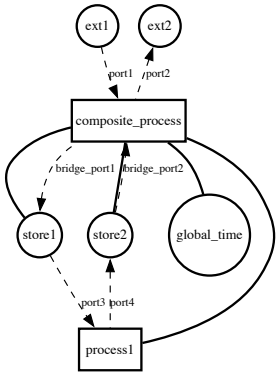

Figure 4: **State tree and diagram of a composite process.** *Left:* a state tree instantiating the declared ports and connecting them to stores. *Right:* the resulting process bigraph visualized with `bigraph-viz`.

3.8 Orchestration

The orchestration pattern used by the `Composite` class follows the spirit of the Discrete Event System Specification (DEVS), in which simulation components can declare their own time advance, and a global orchestrator repeatedly executes the next imminent event. In our setting, these ideas are reinterpreted for typed process–bigraphs: components become process nodes with typed ports, and transitions become type-directed updates over schema/state trees.

3.8.1 Event times and schedule

Each process node $p \in \mathsf{LinkPath}$ carries:

- an *interval* value $\mathsf{interval}(p)$ declaring its nominal time advance;
- an *address* linking it to a handler class in the `link_registry` R_L (whose configured instance is stored at p in the state tree);
- typed input and output ports whose wiring is determined by the `inputs`/`outputs` entries in the state tree.

At runtime, the composite maintains:

- a global time $t \in \mathbb{R}_{\geq 0}$,
- a schedule $t_{\text{next}} : \mathsf{LinkPath} \to \mathbb{R}_{\geq 0}$ assigning each process node its next event time.

Initially, one may set $t_{\text{next}}(p) = t_0 + \mathsf{interval}(p)$ for an initial time t_0.

3.8.2 Small-step orchestration semantics

A configuration is a triple

$$\langle B, t, t_{\text{next}} \rangle,$$

where $B = (\Sigma, x, R_T, R_L)$ is a well-typed Process–Bigraph, t is the current time, and t_{next} is the current schedule.

We define a transition

$$\langle B, t, t_{\text{next}} \rangle \;\longrightarrow\; \langle B', t', t'_{\text{next}} \rangle$$

by the following rules.

(Select) Scheduler step. Let

$$t^\star = \min_{p \in \mathsf{LinkPath}} t_{\text{next}}(p)$$

be the next event time. Let

$$S = \{\, p \in \mathsf{LinkPath} \mid t_{\text{next}}(p) = t^\star \,\}$$

be the set of all processes whose events occur at $t^\star$. We advance the global time to $t' = t^\star$. All processes in S then undergo a *Process Update*, conceptually in parallel when their deltas commute.

(Process Update). For each $p \in S$ — all processes updating at the same event time $t^\star$ — the runtime performs:

1. Perform input projection by reading the state paths listed in the `inputs` field at p, yielding x_{in}.
2. Invoke the configured handler instance stored at path p:

$$\delta_p = \mathsf{update}_p(x_{\text{in}},\, \Delta t),$$

   where $\Delta t = t^\star - t$. The result δ_p is a (possibly composite) delta structured by the output ports of p.
3. Apply δ_p to the state paths listed in the `outputs` field of p using the type engine's apply_τ operators.

Collectively applying all deltas $\{\delta_p \mid p \in S\}$ yields the updated state tree x' and schema tree Σ'. This produces the updated Process–Bigraph $B' = (\Sigma', x', R_T, R_L)$, with internal wiring re-derived from the updated `inputs`/`outputs` fields.

(Reschedule). For each process node p that persists in B', set

$$t'_{\text{next}}(p) = t' + \mathsf{interval}(p),$$

unless overridden by process-specific semantics (e.g. disabling or pausing a process). Process nodes created or removed by structural deltas update the domain of t'_{next} accordingly.

3.8.3 DAG-structured discrete workflows

If all processes in a subsystem satisfy $\mathsf{interval}(p) = 0$, scheduling reduces to a directed acyclic graph (DAG) evaluation over these `step` nodes, with edges determined by data dependencies in the wiring.

Within a single global time t, execution proceeds as follows:

- a step p becomes *eligible* when all input ports updated in this tick have received new values;
- inputs unchanged in this tick do not block execution, allowing partially updated data to propagate;
- execution proceeds in topological order: once a step applies its output deltas, any successor steps whose inputs changed become eligible.

Cycles are disallowed, ensuring deterministic zero-time DAG evaluation within the broader event-scheduling semantics.

Because `Composite` nodes can behave like ordinary processes, an entire composite simulation can itself appear as a single `step` in a larger DAG. This enables end-to-end workflows in which multiple simulations and other tools such as parameter estimation and analysis can be integrated within the same framework.

3.8.4 Recursive composition

Because a `Composite` encapsulates its own scheduler, stores, and internal processes, it acts externally as a process but internally executes a complete process–bigraph. For temporal processes inside the composite, the internal scheduler maintains its own event queue, advancing the front of that schedule in time and updating the embedded state and schema as structural deltas create, remove, or move process nodes. The composite therefore continually re-tracks all available internal processes, their paths, and their next update times. After each internal update, the composite advances its own event time according to its interval field, and outward-facing results are propagated through its interface ports via the `bridge`. In this way, temporal dynamics, DAG-style workflows, and structural graph rewrites coexist naturally within a single hierarchical system under uniform schema-based, typed-link semantics.

3.9 Emitters and result capture

Emitters are special `step` processes whose role is observational of the simulation state for saving results. An emitter declares typed input ports, but no semantic outputs back into the simulation state. At execution time, the emitter reads values from its input ports and records them into an external or side-channel store for later retrieval, analysis, or visualization.

Operationally, an emitter participates in orchestration exactly like any other step process: when scheduled, it performs input projection, invokes its handler, and produces a delta. However, the delta produced by an emitter is it writing to a recording backend (e.g. in-memory buffers, disk, streaming logs), rather than as an update to the schema/state tree. Emitters therefore do not introduce new data dependencies into the process–bigraph and cannot affect subsequent process execution.

4 Spatio–Flux Instantiation

This section gives a concrete example of the Process–Bigraph applied to the Spatio–Flux example from the main text. We begin by introducing some of Spatio–Flux's processes in 4.1. These are then integrated for the reference demonstration in 4.2, with all the required config parameters and wiring. All documents and tests for this example are available in the public `spatio-flux` repository, as well as many other novel compositions.

The repository also contains an automated test suite that serves as a living catalog of compositional patterns. Tests are defined as executable composite documents and are designed to be extended as new process families and adapters are added. Because the suite is intended to evolve alongside the library, individual test configurations may be refined over time while preserving their role as reproducible examples and semantic checks.

4.1 Spatio–Flux processes

Spatio–Flux instantiates the Process Bigraph semantics through a small set of reusable processes that can be wired into the `fields` and/or `particles` stores. Each process is a `process` or `step` node whose schema specifies typed input and output ports and whose state-tree entry stores its `address`, `config`, and wiring to concrete store paths. Upon execution, each process receives an input projection, computes an update delta tree, and the orchestration engine applies it.

Table 1 summarizes the main process families, organized by modeling concern and linked to the corresponding subsections below. These families progress from local metabolic dynamics, to spatial transport, to particle motion, to adapter processes that couple representations, and finally to structural graph rewrites that modify population topology. The detailed process "cards" that follow provide a concrete reference for each process's interface, configuration, and semantics.

The process cards were generated semi-automatically from process schemas, docstrings, and reference configurations, with light manual editing to improve presentation. A more streamlined card-generation pipeline may be integrated into `spatio-flux` in future releases, enabling documentation and test-suite examples to be rendered from the same source artifacts.

Process family	**Processes**
Metabolic	`DynamicFBA`, `MonodKinetics`, `SpatialDFBA`
Field transport	`DiffusionAdvection`
Particle movement	`BrownianMovement`, `PymunkParticleMovement`
Particle–field adapters	`ParticleExchange`, `ParticleTotalMass`
Structural and boundary	`ParticleDivision`, `ManageBoundaries`

Table 1: **Spatio–Flux process families.** Each family links to a detailed subsection describing its processes and semantics.

4.1.1 Local metabolic processes

Metabolic processes operate on individual lattice sites, which store derived concentrations. Each metabolic process reads local substrate concentrations and biomass, computes reaction rates, and writes back typed count deltas to the same site, ensuring compatibility with the `conc_counts` update semantics. Because the metabolic processes share a common typed interface, they can be interchanged freely at any lattice location.

Monod kinetics (ODE) local metabolic

The `monod_kinetics` process provides a saturating-rate ODE for uptake and secretion at a single lattice site. It implements the `metabolism` interface with a compact configuration in terms of reactions and kinetic parameters.

Interface

`process monod_kinetics`		
Name	**Direction**	**Type**
biomass	input	concentration
substrates	input	map[concentration]
biomass	output	counts
substrates	output	map[counts]

Parameters

Parameter	Type	Default	Role
reactions	map[reaction]	{`assimilate_glucose`, `maintenance_turnover`}	Reaction definitions mapping `reactant` and `product` species for each Monod term.
kinetic_params	map[tuple[float,float]]	`glucose` : $(0.5, 0.01)$; `mass` : $(1.0, 0.001)$	Per-species Monod parameters $(K_m, V_{\max})$ for substrate uptake and mass turnover.

Semantics and equations. For each configured Monod term with reactant R (a substrate concentration c_R, or biomass m if $R = \texttt{mass}$) and product P, the site-local rate is

$$v_R \;=\; \frac{V_{\max,R}\, c_R}{K_{m,R} + c_R}.$$

Over timestep Δt, the process returns stoichiometric deltas in `counts` form: reactant is consumed $\Delta N_R = -v_R \Delta t$, biomass increases when $P = \texttt{mass}$ via $\Delta m = v_R \Delta t$, and any product substrate S receives $\Delta N_S = +v_R \Delta t$.

Dynamic FBA (network-level metabolism) local metabolic

The `dfba` process uses a full metabolic network model and computes feasible instantaneous fluxes using `cobraPy`. It implements the same `metabolism` interface as `monod_kinetics`, but with a configuration that points to a COBRA model and specifies how substrates are mapped to exchange reactions.

Interface

`process dfba`		
Name	**Direction**	**Type**
biomass	input	concentration
substrates	input	map[concentration]
biomass	output	counts
substrates	output	map[counts]

Parameters

Parameter	Type	Default	Role
model_file	string	required	Path to the COBRA metabolic model used for FBA.
kinetic_params	map[tuple[float,float]]	required	Per-substrate Michaelis–Menten parameters $(K_m, V_{\max})$ used to set uptake bounds.
substrate_update_reactions	map[string]	required	Map from substrate identifiers to exchange reaction IDs in the model.
bounds	map[bounds]	required	Optional overrides for exchange-reaction lower/upper bounds.

Semantics and equations. For each substrate s with concentration c_s, an uptake bound is computed via Michaelis–Menten kinetics and applied to the corresponding exchange reaction:

$$v_s^{\text{uptake}} \;=\; -\frac{V_{\max,s}\, c_s}{K_{m,s} + c_s}.$$

Given bounds $\ell \le v \le u$, the COBRA model solves an instantaneous FBA problem (typically maximizing growth), yielding an optimal growth rate μ and exchange fluxes v_s. The returned `counts`-typed deltas over Δt are

$$\Delta m \;=\; \mu\, m\, \Delta t, \qquad \Delta N_s \;=\; v_s\, m\, \Delta t,$$

with substrate deltas clamped to preserve non-negativity under `conc_counts` semantics.

4.1.2 Field movement

Field movement processes operate on the two-dimensional arrays of typed `conc_counts` sites stored in the `fields` map. Each species occupies a lattice of concentrations, and the field processes update these concentrations by computing spatial fluxes and returning typed deltas that the `conc_counts_apply` operator converts into consistent `counts` and `concentration` updates. Because the interface is typed, field processes integrate seamlessly into the Process–Bigraph semantics: inputs project

each species array, the handler computes spatial fluxes, and outputs return per-site deltas under the `fields` port.

Diffusion–advection field movement

The `diffusion` process updates the concentration fields of all substrates on the spatial lattice using an explicit advection–diffusion scheme. It reads the current concentration arrays for each species and returns typed deltas that modify the same arrays in place.

Interface

`process diffusion`		
Name	**Direction**	**Type**
fields	input	map[array[concentration]]
fields	output	map[array[concentration]]

Parameters

Parameter	**Type**	**Default**	**Role**
n_bins	tuple[integer,integer]	required	Number of lattice cells in (x, y).
bounds	tuple[float,float]	required	Physical domain size in (L_x, L_y).
default_diffusion_rate	float	`1e-1`	Default diffusion coefficient D used if no species-specific value is given.
diffusion_coeffs	map[float]	none	Per-species diffusion coefficients overriding the default.
advection_coeffs	map[tuple[float,float]]	none	Per-species advection velocities (v_x, v_y).
max_dt	float	`1e-1`	Upper bound on internal diffusion timestep.
cfl_adv	float	`0.5`	CFL stability factor for the advection term.
clip_negative	boolean	`true`	If true, negative concentrations are clipped to zero.

Semantics and equations. For each species field C_s, concentrations evolve according to

$$\partial_t C_s \;=\; D_s \nabla^2 C_s \;-\; \mathbf{v}_s \cdot \nabla C_s.$$

The handler selects a stable internal timestep δt (CFL-limited and capped by `max_dt`) and advances the field with $n = \lceil \Delta t / \delta t \rceil$ explicit Euler substeps using standard finite-difference stencils. The emitted output is a per-species delta array

$$\Delta C_s = C_s^{\text{final}} - C_s^{\text{initial}},$$

optionally clipped to enforce non-negative concentrations.

4.1.3 Particle movement processes

Particle movement processes operate on the `particles` store, a typed map from particle identifiers to particle records. The schema distinguishes base `particle` records from `complex_particle` records that also include mechanical attributes (shape, velocity, inertia, friction, elasticity, radius). Each movement process is a typed `process` node whose ports are wired to `particles` and whose update method returns deltas that modify particle attributes and, when needed, perform structural rewrites on the particle map.

Brownian movement particle movement

The `brownian_movement` process models overdamped drift–diffusion of particles inside a rectangular domain. It acts on base `particle` records and updates only the `position` field of each surviving particle, with optional boundary birth/death.

Interface

`process brownian_movement`		
Name	**Direction**	**Type**
particles	input	map[particle]
particles	output	map[particle]

Parameters

Parameter	Type	Default	Role
bounds	tuple[float,float]	required	Spatial domain size used to clamp particle positions.
diffusion_rate	float	`1e-1`	Brownian diffusion coefficient D.
advection_rate	tuple[float,float]	(0.0, 0.0)	Uniform drift velocity (v_x, v_y).

Semantics and equations. Each particle position $X_i \in \mathbb{R}^2$ is updated over Δt using an Euler–Maruyama drift–diffusion step:

$$X_i \leftarrow X_i + \mathbf{v}\,\Delta t + \sqrt{2D\,\Delta t}\,\xi_i, \qquad \xi_i \sim \mathcal{N}(0, I_2).$$

The process emits typed deltas that update only the `position` field of each particle. Particles that cross configured absorbing boundaries may be removed via structural `_remove` deltas when boundary-driven rewrites are enabled; otherwise, positions are unconstrained by the process itself.

Newtonian particles particle movement

The `newtonian_particles` process refines particle mechanics by treating each particle as a rigid body in a 2D physics engine (Pymunk). It acts on `complex_particle` records with explicit mechanical attributes and updates both positions and velocities, with optional boundary birth/death.

Interface

`process newtonian_particles`

Name	Direction	Type
particles	input	map[particle]
particles	output	map[particle]

Parameters

Parameter	Type	Default	Role
bounds	tuple[float,float]	(500.0, 500.0)	Physical domain size for particle motion.
diffusion_rate	float	`1e-1`	Effective mechanical diffusion strength (jitter scale).
advection_rate	tuple[float,float]	(0.0, 0.0)	Global drift velocity applied to all particles.
substeps	integer	`100`	Number of Pymunk substeps per external timestep.
damping_per_second	float	`0.98`	Global velocity damping per second.
gravity	float	`-9.81`	Vertical gravitational acceleration.
friction	float	`0.8`	Contact friction coefficient.
elasticity	float	`0.0`	Contact elasticity (bounciness).
jitter_per_second	float	`1e-2`	Standard deviation of random impulse per second.
barriers	list[map]	`[]`	Optional internal obstacles/walls definitions.
wall_thickness	float	`100.0`	Thickness of outer domain walls.

Semantics and equations. Particles are simulated as rigid bodies governed by Newtonian mechanics,

$$m\,\frac{dV}{dt} = F_{\text{net}}, \qquad \frac{dX}{dt} = V,$$

implemented via a 2D physics engine with configured gravity, damping, contact mechanics, and stochastic jitter. Over an external timestep Δt, the engine advances in multiple substeps, after which the process emits deltas updating particle positions, velocities, and other mechanical attributes. Boundary crossings may additionally trigger `_add`/`_remove` rewrites on the particle map.

4.1.4 Particle–field adapters

Particle–field exchange adapter

`particle_exchange` is a `step` node that acts as an adapter between the continuous particle representation and the discrete field lattice. It introduces no new physics of its own; instead, it (i) samples lattice concentrations at particle locations into per-particle `local` state, and (ii) applies per-particle `exchange` *amounts* into the appropriate lattice bins as *concentration* updates while preserving mass balance by converting amounts to concentration deltas using the bin volume implied by the spatial discretization.

Interface

`step particle_exchange`		
Name	**Direction**	**Type**
particles	input	map[particle]
fields	input	fields
particles	output	map[particle]
fields	output	fields

Parameters

Parameter	**Type**	**Default**	**Role**
bounds	tuple[float,float]	required	Domain bounds (L_x, L_y) used to map particle positions to lattice indices.
n_bins	tuple[integer,integer]	required	Number of lattice cells (n_x, n_y); must match the `fields` grid.
depth	float	required	Assumed depth of the 2D lattice used to compute per-bin volume.

Semantics and equations. Each particle position $X_i = (x_i, y_i)$ is mapped to a lattice cell (c_i, r_i) by uniform binning over $[0, L_x] \times [0, L_y]$. Let $C_m(c, r)$ denote the field concentration of species m at that cell. The adapter updates each particle's `local` snapshot by sampling:

$$\texttt{local}_{i,m} \leftarrow C_m(c_i, r_i),$$

emitting either an initial `_add` (first sample) or a delta update thereafter.

To preserve mass balance between particle-local `exchange` *amounts* and field *concentrations*, the adapter converts each queued per-particle exchange amount $\Delta A_{i,m}$ into a concentration delta in the target bin using the bin volume

$$V_{\text{bin}} \;=\; \Delta x\,\Delta y\,\text{depth}, \qquad \Delta x = \frac{L_x}{n_x},\ \Delta y = \frac{L_y}{n_y}.$$

For each species m, it applies

$$\Delta C_m(c_i, r_i) \mathrel{+}= \frac{\Delta A_{i,m}}{V_{\text{bin}}}.$$

Finally, consumed exchange entries are reset to zero via particle deltas, ensuring one-shot transfer semantics.

Aggregate particle mass adapter

The `aggregate_mass` process is a `step` node that acts as an adapter between compositional internal particle state and a single scalar mass used by downstream processes. It introduces no new dynamics of its own; instead, it aggregates multiple per-particle `sub_masses` contributions into a single `total_mass` value, providing a well-defined interface between internal metabolic structure and particle-level structural rules.

Interface

`step aggregate_mass`		
Name	**Direction**	**Type**
sub_masses	input	map[mass]
total_mass	output	overwrite[mass]

Parameters

Parameter	**Type**	**Default**	**Role**

Semantics and equations. Let $\{\ m_k\ \}_{k\in K}$ denote the values stored in the particle-local `sub_masses` map. The adapter computes the total particle mass as a simple aggregation,

$$m_{\text{total}} \;=\; \sum_{k\in K} m_k,$$

ignoring any non-numeric entries defensively. The resulting scalar is emitted as an `overwrite` update to `total_mass`, replacing any previous value. By separating internal mass accumulation from its downstream interpretation, this adapter allows particle-level structural processes (e.g. division, removal, or mechanical scaling) to depend on a single scalar signal while preserving compositional internal structure.

4.1.5 Structural and Boundary – Graph rewrites

Particle division graph rewrite

The `particle_division` process is a `step` node whose primary role is to demonstrate graph rewrites on the `particles` store. It has a purely particle-based interface and performs no continuous time integration; instead, it rewrites the particle map whenever a mass threshold is exceeded.

Interface

`step particle_division`		
Name	**Direction**	**Type**
particles	input	map[particle]
particles	output	map[particle]

Parameters

Parameter	**Type**	**Default**	**Role**
division_mass_threshold	float	`0.0`	Mass threshold above which particles are eligible to divide (disabled if ≤ 0).
division_jitter	float	`1e-3`	Fraction of a reference length used as jitter radius to place daughter particles.

Semantics and equations. For each particle with mass $m_i \geq m_{\text{thr}}$, the process performs a graph rewrite: the parent particle is removed and two daughters are inserted. Daughter positions are displaced symmetrically by a small random offset, and masses split the parent mass,

$$m_{i,1} = m_{i,2} = \tfrac{1}{2} m_i.$$

All changes are emitted as `_remove` and `_add` deltas on the `particles` map.

Manage boundaries (reflect & absorb) boundary policy

The `manage_boundaries` step enforces boundary policies on particle *absolute* positions. It reads the `particles` map, applies absorbing removal for configured sides, reflects all other out-of-range coordinates back into the domain interior, and can spawn new particles at selected boundaries according to a Poisson birth process.

Interface

`step manage_boundaries`		
Name	**Direction**	**Type**
particles	input	map[particle]
process_interval	input	float
particles	output	map[particle]

Parameters

Parameter	Type	Default	Role
bounds	tuple[float,float]	required	Domain size (L_x, L_y) defining hard bounds $[0, L_x] \times [0, L_y]$.
add_rate	float	`0.0`	Birth event rate (1/sec) used to sample boundary-spawn events.
boundary_to_add	list[boundary_side]	`["top"]`	Sides where new particles may be spawned.
boundary_to_remove	list[boundary_side]	`[]`	Absorbing sides; particles crossing a configured side are removed.
buffer	float	`1e-6`	Numerical buffer used to reflect into the interior to avoid floating-point sticking on walls.
mass_range	tuple[float,float]	`INITIAL_MASS_RANGE`	Mass range used when generating newborn particles.

Semantics and equations. Let the hard domain be $[0, L_x] \times [0, L_y]$ and the interior reflection domain be $[\,\epsilon,\ L_x - \epsilon\,] \times [\,\epsilon,\ L_y - \epsilon\,]$ with $\epsilon =$ `buffer`. For each particle id i with absolute position $X_i = (x_i, y_i)$:

1. *Absorbing removal.* If X_i violates a configured absorbing side (e.g. $x_i < 0$ when `left` is absorbing, $x_i > L_x$ when `right` is absorbing, etc.), emit a structural `_remove` delta for i.
2. *Reflection.* Otherwise, any out-of-range coordinate is reflected back into the interior interval. In one dimension, reflection into $[lo, hi]$ is computed by

$$\text{reflect}(x; lo, hi) \;=\; \begin{cases} lo + y, & y \le (hi - lo) \\ hi - (y - (hi - lo)), & y > (hi - lo) \end{cases} \qquad y = (x - lo) \bmod 2(hi - lo),$$

and applied componentwise:

$$(x_i', y_i') = \big(\text{reflect}(x_i; \epsilon, L_x - \epsilon),\ \text{reflect}(y_i; \epsilon, L_y - \epsilon)\big).$$

Emit an *absolute* position update delta for particle i: $\Delta_i = \{\texttt{position} : (x_i', y_i')\}$.

Boundary births. Let $\Delta t =$ `process_interval` and birth rate $\lambda =$ `add_rate`. For each boundary $b \in$ `boundary_to_add`, sample an event with probability

$$p_{\text{birth}} = 1 - e^{-\lambda \Delta t}.$$

On success, generate a new id j, choose an absolute spawn position X_j on side b, sample mass from `mass_range`, and emit a structural `_add` delta inserting the new particle record.

The output under `particles` therefore combines (i) attribute deltas for absolute position updates, and (ii) structural edits `_remove`/`_add` implementing absorbing boundaries and Poisson births.

4.2 Particle–COMETS composition

This subsection lists the concrete wiring and parameter values used for the reference composite shown in the main text, instantiated as a single `Composite` (`particle_comets`). The `spatio-flux` test suite includes a designated reference composite for regression and demonstration, but its exact configuration may evolve as new processes become available and improved models and parameters are integrated.

`composite particle_comets`

diffusion field transport

`process diffusion`

Wiring

Port	Direction	Store path
`fields`	input	`fields`
`fields`	output	`fields`

Properties

Property	Value
address	`local.DiffusionAdvection`
interval	`1.0`

Config

Key	Value
bounds	[50.0, 50.0]
n_bins	[10, 10]
default_diffusion_rate	0.1
default_diffusion_dt	0.1
max_dt	0.1
cfl_adv	0.5
clip_negative	true
diffusion_coeffs.glucose	0.1
diffusion_coeffs.acetate	0.1
diffusion_coeffs.dissolved biomass	0.1
advection_coeffs.glucose	[0.0, 0.0]
advection_coeffs.acetate	[0.0, 0.0]
advection_coeffs.dissolved biomass	[0.0, 0.0]
boundary_conditions.default.x.type	periodic
boundary_conditions.default.y.type	neumann
boundary_conditions.glucose.top.type	dirichlet
boundary_conditions.glucose.top.value	5.0
boundary_conditions.acetate.bottom.type	dirichlet
boundary_conditions.acetate.bottom.value	5.0

composite particle_comets

monod_kinetics[i,j] local metabolic

process monod_kinetics

Wiring

Port	Direction	Store path
substrates.glucose	input	fields.glucose[i,j]
substrates.acetate	input	fields.acetate[i,j]
biomass	input	fields.dissolved biomass[i,j]
substrates.glucose	output	fields.glucose[i,j]
substrates.acetate	output	fields.acetate[i,j]
biomass	output	fields.dissolved biomass[i,j]

Properties

Property	Value
address	local.MonodKinetics
interval	1.0

Config

Key	Value
reactions.assimilate_glucose.reactant	glucose
reactions.assimilate_glucose.product	mass
reactions.assimilate_glucose.km	1.0
reactions.assimilate_glucose.vmax	0.02
reactions.assimilate_glucose.yield	0.15
reactions.overflow_to_acetate.reactant	glucose
reactions.overflow_to_acetate.product	acetate
reactions.overflow_to_acetate.km	1.0
reactions.overflow_to_acetate.vmax	0.2
reactions.overflow_to_acetate.yield	0.85
reactions.maintenance_turnover.reactant	mass
reactions.maintenance_turnover.product	detritus
reactions.maintenance_turnover.km	1.0
reactions.maintenance_turnover.vmax	0.001
reactions.maintenance_turnover.yield	1.0

composite particle_comets

newtonian_particles particle movement

process newtonian_particles

Wiring

Port	Direction	Store path
particles	input	particles
particles	output	particles

Properties

Property	Value
address	local.PymunkParticleMovement
interval	1.0

Config

Key	Value
bounds	[50.0, 50.0]
substeps	100
gravity	-1.0
damping_per_second	0.95
jitter_per_second	0.01
diffusion_rate	0.1
advection_rate	[0.0, 0.0]
friction	0.9
elasticity	0.1
wall_thickness	100.0
barriers	[]
new_particle_radius_range	[0.5, 2.0]
new_particle_mass_range	[1.0, 1.0]
new_particle_density	0.015
new_particle_speed_range	[0.0, 0.0]
circle_density	0.015
segment_radius	10.0
segment_density	0.02
segment_radius	10.0
n_bins	[1, 1]

composite particle_comets

particle_exchange particle–field coupling

step particle_exchange

Wiring

Port	Direction	Store path
particles	input	particles
fields	input	fields
particles	output	particles
fields	output	fields

Properties

Property	Value
address	local.ParticleExchange
interval	1.0

Config

Key	Value
bounds	[50.0, 50.0]
n_bins	[10, 10]
apply_mass_balance	false
rates_are_per_time	true

composite particle_comets

enforce_boundaries boundary management

step enforce_boundaries

Wiring

Port	Direction	Store path
particles	input	particles
process_interval	input	newtonian_particles.interval
particles	output	particles

Properties

Property	Value
address	local.ManageBoundaries
interval	1.0

Config

Key	Value
bounds	[50.0, 50.0]
buffer	0.0001
clamp_survivors	true
mass_range	[0.001, 1.0]
add_rate	0.0
boundary_to_remove	[]
boundary_to_add	["top","top","bottom","left","right"]

composite particle_comets

particle_division graph rewrite

step particle_division

Wiring

Port	Direction	Store path
particles	input	particles
particles	output	particles

Properties

Property	Value
address	local.ParticleDivision
interval	1.0

Config

Key	Value
division_mass_threshold	0.4
division_jitter	0.001
sub_mass_split	equal
submass_split_mode	random

composite particle_internal

ecoli_1 dFBA particle metabolic

process ecoli_1 dFBA

Wiring

Port	Direction	Store path
substrates	input	local
biomass	input	sub_masses.ecoli_1
substrates	output	exchange
biomass	output	sub_masses.ecoli_1

Properties

Property	Value
address	local.DynamicFBA

Config

Key	Value
model_file	textbook
kinetic_params.glucose	[0.1, 2.0]
kinetic_params.acetate	[1.0, 0.1]
substrate_update_reactions.glucose	EX_glc__D_e
substrate_update_reactions.acetate	EX_ac_e
bounds.ATPM.lower	1.0
bounds.ATPM.upper	1.0
bounds.EX_o2_e.lower	-2.0

composite particle_internal

ecoli_2 dFBA particle metabolic

process ecoli_2 dFBA

Wiring

Port	Direction	Store path
substrates	input	local
biomass	input	sub_masses.ecoli_2
substrates	output	exchange
biomass	output	sub_masses.ecoli_2

Properties

Property	Value
address	local.DynamicFBA

Config

Key	Value
model_file	textbook
kinetic_params.glucose	[1.0, 0.1]
kinetic_params.acetate	[0.01, 1.0]
substrate_update_reactions.glucose	EX_glc__D_e
substrate_update_reactions.acetate	EX_ac_e
bounds.ATPM.lower	1.0
bounds.ATPM.upper	1.0
bounds.EX_o2_e.lower	-2.0